

Here we go 🐛 —a Typora-ready consolidation doc, with:

1. Plain explanation of **id vs class**
2. A **minimal CSS + HTML pair**
3. A second round of **background property notes**
4. Updated CSS with a **fixed sky background + transparent rounded cards**

You can paste this whole thing into Typora as

```
W3_id_vs_class_background.md.
```

Week -2

id vs class & background basics

1. What is the difference between **id** and **class**?

1.1 Simple idea

- **id**
 - Used to **identify one unique element** on the page.
 - Like a **passport number**: one person, one number.
 - In HTML: `id="main-wrapper"`
 - In CSS: `#main-wrapper { ... }`
- **class**
 - Used to **group elements that share the same style**.
 - Like a **team name or category**: many people can belong to the same team.

- In HTML: `class="card"`
- In CSS: `.card { ... }`

1.2 Quick comparison

Feature	<code>id</code>	<code>class</code>
Uniqueness	Should be unique	Can be used on many elements
CSS selector	<code>#idName</code>	<code>.className</code>
Typical use	Main containers, anchors	Repeated cards, buttons, labels
Reusability	Normally 1 per page	Reused freely

2. Mini layout: one main container + 3 floating cards

We will build:

- A page where:
 - The `<body>` has a **sky background** (fixed, full screen).
 - One main container `<div id="page-wrapper">`
 - Inside it, **three small boxes** `<div class="card">` that:
 - Float from **left to right**,
 - Each contains an image + placeholder text,
 - Have a **semi-transparent background** with **rounded corners**.

We will use two files:

- `index.html`
- `styles.css`

5. Background property: notes for the learner

Now we focus on the **background** of elements. We can control:

- Just the **background color**.
- An **image** as background.
- Whether it repeats, where it is positioned, how big it is, if it is fixed, etc.
- Or we can use the **shorthand** `background` to set several things at once.

5.1 Atomic background properties

Property	What it does (simple words)	Example
<code>background-color</code>	Fills the element with a solid color	<code>background-color: #f0f4ff;</code>
<code>background-image</code>	Sets an image as background	<code>background-image: url("sky.jpg");</code>
<code>background-repeat</code>	Controls if the image is tiled (repeated) or not	<code>background-repeat: no-repeat;</code>
<code>background-position</code>	Where the image is anchored inside the element	<code>background-position: center center;</code>
<code>background-size</code>	How large the image is (original size, cover, contain, etc.)	<code>background-size: cover;</code>
<code>background-attachment</code>	If the background scrolls with content or stays fixed in viewport	<code>background-attachment: fixed;</code>

5.2 The shorthand `background`

The shorthand lets us write several of these in one line:

```
/* Color + image + repeat + position + attachment in one line
*/
background: #e0f7ff url("images/sky.jpg") no-repeat center
center fixed;
background-size: cover; /* size is usually written separately
*/
```

Reading that:

- `#e0f7ff` → fallback color (behind the image, or if image fails to load)
- `url("images/sky.jpg")` → the sky photo
- `no-repeat` → do not tile the image
- `center center` → center it horizontally and vertically
- `fixed` → attach background to viewport (gives “parallax” feeling when you scroll)

Note on Image Styling Choice for `.card img`

In this project the source images have **irregular, idiosyncratic sizes**, which can easily produce uneven card heights or distorted images if a strict fixed width–height pair is used. To maintain a **coherent visual rhythm** across the three floating cards—while still allowing the layout to adapt responsibly—we adopt the following balanced rule:

```
.card img {  
  width: 100%;  
  max-width: 288px;  
  aspect-ratio: 3 / 2;  
  object-fit: cover;  
  display: block;  
  margin-bottom: 10px;  
}
```

This configuration offers three advantages:

1. **Responsive scaling**

`width: 100%` allows the image to adapt to the card’s width on different screens, while `max-width: 288px` preserves the clean, compact card style expected in this exercise.

2. Consistent proportions without distortion

`aspect-ratio: 3 / 2` defines a stable rectangular shape for all images.

Combined with `object-fit: cover`, the image fills that space naturally without stretching, bending, or appearing squeezed.

3. Uniform visual presentation despite varied originals

Even if the source files have different shapes or orientations, the final displayed cards maintain a **harmonised look**, which helps the learner focus on layout concepts rather than accidental image irregularities.

Overall, this rule provides a **controlled but flexible** approach: the design remains stable, aesthetically aligned across cards, and visually clear on all devices, while still accommodating images with unpredictable original dimensions.

Expansion of the Project: Botanical Cards with Nested Lists

This extension introduces **additional cards** containing fictional flowers and their associated characteristics. Each card will display an image and a structured block of information using a combination of **ordered lists**, **unordered lists**, and **nested lists**. The purpose is to practise semantic HTML, list structuring, and consistent layout styling inside `.card` components.

Structure of Information per Card

Each flower card follows this conceptual organisation:

1. Top-level ordered list (`<ol>`)

◦ Item 1: Latin name

Inside this item, we place a **nested unordered list** (`<ul>`) containing:

- English name
- Spanish name

- Catalan name

2. Item 2: Seasonal data

- “Peak blooming season” (kept inside the ordered list sequence)

3. Item 3: Habitat / location

- Brief description (also within the ordered sequence)

4. Item 4: Allergy information

Inside this item, we add:

- An unordered nested list with the **fictional allergy effects** (e.g., “mild ocular irritation”, “ticklish sneezing reflex”, etc.)

This structure ensures that:

- The **Latin name** remains the primary, formal identifier (Item 1).
- Vernacular names in three languages are grouped neatly under it.
- Environmental and seasonal factors follow a logical descriptive sequence.
- Allergy aspects—being multiple and variable—fit naturally inside a nested unordered list.

The resulting layout is semantically clear, visually tidy, and easy to style within the existing `.card` component.

Styling Ordered and Unordered Lists

To visually distinguish the hierarchical levels inside each flower card:

- The **ordered list** (`<o1>`) items are styled in **bold**, emphasising the main structural descriptors:
Latin name, season, habitat, and allergy notes.

- The **unordered list** (`<ul>`) nested under them uses **custom flower bullets**, reinforcing the botanical theme and helping the eye quickly separate secondary information (vernacular names, allergy effects).
Instead of the default circle/disc bullets, each `<li>` displays a **flower symbol**.

This contrast (bold main items + floral secondary bullets) makes the informational layers clearer and adds an aesthetic coherence that supports the spring theme of the project.